

PROJETO FINAL

Reconhecimento de Atividades Humanas com TinyML

ESP32-S3 + MPU6050 + TensorFlow Lite Micro

UC IA Embarcada e Modelos Compactos

6 atividades

modelo INT8

Wokwi validado

TFLM

Gabriel Monteiro de Souza
Renan Cardoso dos Santos
Rodrigo Teles Dondé

81,13%

accuracy INT8

4.240 B

modelo INT8

21

testes aprovados

6/6

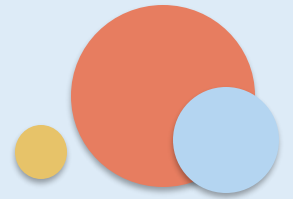
REPLAY no Wokwi

Proposta do projeto

Classificar atividades humanas localmente, com um modelo compacto, quantizado e executando no ESP32-S3.

Problema e objetivo

Por que usar IA embarcada para reconhecer atividades humanas?



Problema

Seis atividades precisam ser diferenciadas a partir de sinais de aceleração e rotação. Classes diferentes podem compartilhar amplitudes semelhantes.

WALKING

caminhar

STANDING

em pé

Por que IA?

O modelo aprende a combinação entre os seis eixos e permite inferência local, com baixa latência e sem depender da nuvem.

SITTING

sentar

DOWNSTAIRS

descer

Objetivo

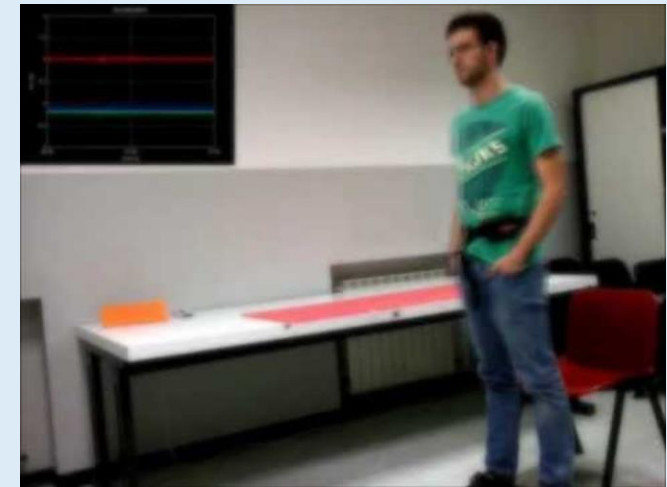
Desenvolver uma aplicação de IA Embarcada para um problema de classificação de atividades humanas, com um modelo compacto, quantizado e executado de forma simulada em um MCU ESP32-S3.

UPSTAIRS

subir

LAYING

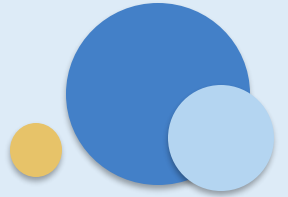
deitado



Activity Recognition Experiment Using Smartphone Sensors.

Dados e hardware

Do UCI HAR ao ESP32-S3 com MPU6050



Dataset - UCI HAR

- 30 voluntários (19–48 anos), 6 atividades
- Galaxy S II na cintura: acelerômetro (g) + giroscópio (rad/s), 3 eixos
- **Amostragem:** 50 Hz constante
- **Nyquist:** capta até 25 Hz; o movimento humano fica abaixo de 20 Hz
- **Resolução:** float32; bits do sensor não documentados
- **Filtro de ruído:** mediana + Butterworth passa-baixa de 3ª ordem, 20 Hz
- **Gravidade:** Butterworth passa-baixa 0,3 Hz separa body_acc
- **Janelamento:** 128 amostras = 2,56 s

Hardware embarcado

- ESP32-S3
- MPU6050 simulado no Wokwi
- I2C nos GPIO 8/9
- TensorFlow Lite Micro
- Inferência executada localmente

Divisão sem vazamento

Treino: 5.551 janelas
Validação: 1.801 janelas
Teste: 2.947 janelas

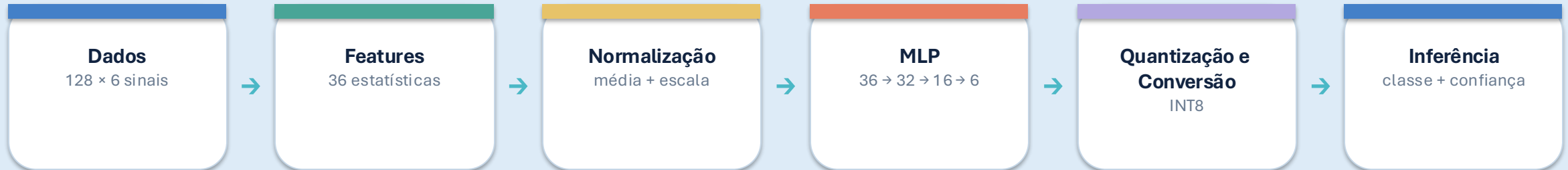
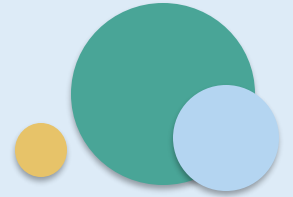
Os grupos de treino, validação e teste usam voluntários diferentes.

Ponto metodológico importante

A accuracy de 81,13% refere-se ao teste UCI HAR. O modo LIVE comprova a execução da pipeline com o MPU6050, mas não substitui uma validação estatística com dados próprios.

Pipeline de ponta a ponta

A mesma lógica é reproduzida no treinamento e no firmware



36 features por janela

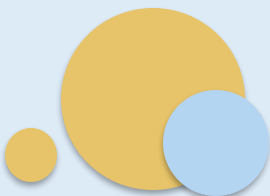
Para cada canal: média, desvio-padrão, RMS, mínimo, máximo e energia.

Equivalência validada

21 testes aprovados para features, quantização, golden vectors, janelas e artefatos Python/C.

Treinamento do modelo

MLP compacta treinada em FP32 com dataset público (UCI HAR)



Objetivo: Escolher a menor rede que resolve o problema.

Arquitetura da MLP



36 entradas: 6 canais do sensor ([total_acc_; body_gyro_] x 3eixos) x 6 estatística

Cálculo de Parâmetros

- ❑ **Qtd. de Parâmetros (P)** = (entradas x neurônios) + neurônios = **1.814**
- ❑ **Tamanho (bytes)** = parâmetros x bytes por parâmetro
 - **Float32** = $1.814 \times 4 = 7,1$ KB ****referência ganho de compressão**
 - **Int8** = $1.814 \times 1 = 1,8$ KB ****cabe com folga (TinyML < 256KB)**

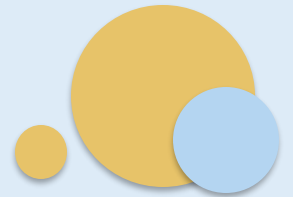
Como foi treinado

Item	Valor	Descrição
Normalização	z-score $(x - \mu) / \sigma$, com μ e σ só do treino	Põe as 36 features na mesma escala. μ e σ vão como constantes para o firmware (model_params.h)
Otimizador / loss	Adam (lr 0,001) / sparse categorical cross-entropy, batch 64	Padrão para multiclasse com rótulos inteiros
Validação	Por voluntário (GroupShuffleSplit), seed 42	Janelas da mesma pessoa são parecidas. Misturar inflaria a accuracy
Early stopping	val_loss, paciência 20 → melhor época 13 de 33	Para quando a validação para de melhorar e restaura os melhores pesos. Evita overfitting
Referência FP32	81,27% no teste	É a régua para medir quanto a compressão custa

Se em float32 o modelo já cabe na placa (~7 KB), por que ainda comprimir?

Conversão e compressão

Quantização INT8 completa e impacto mínimo no desempenho

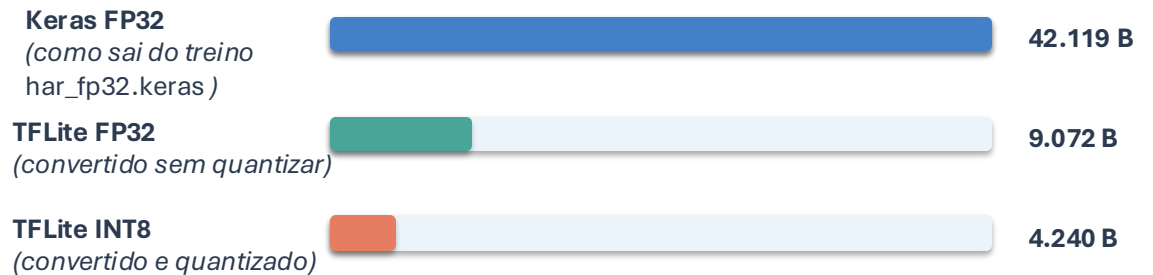


Objetivo: Mostrar como o modelo foi comprimido (método), quanto diminuiu (resultado) e que ele cabe na memória do ESP32-S3 (orçamento).

Como foi quantizado

- 1 PTQ full integer INT8**
Quantização pós-treino: treina em float32 e quantiza depois. TFLITE_BUILTINS_INT8 exige que todas as operações sejam int8.
- 2 Sem pruning**
Zera pesos pequenos. Quantizar deve ser feito primeiro, com 4,2 KB, não há necessidade de reduzir o número de parâmetros.
- 3 Sem destilação**
Arquitetura do modelo de tamanho pequeno, sem necessidade de Destilação.

Tamanho dos artefatos



TFLite FP32 → INT8: redução de 53,26%

Por que 53% e não os 75% teóricos?

Pesos + vieses: 7.256 B → ≈ 1.976 B (-73%; o bias fica em int32).
Arquivo .tflite 9.072 B → ≈ 4.420 B (-53%; tamanho do arquivo).

4,2 KB

O modelo cabe na memória Flash?

Flash: modelo INT8, ≈ 1,6% da meta TinyML de 256 KB (16 MB na placa)

68 B

Quanto de RAM a inferência exige no pior momento?

SRAM: pico de ativação estimado (36 + 32 valores int8)

860 B

Quanto o TFLite Micro usou de fato?

Medido no boot: 68 B de ativações + estruturas do interpretador. Usa ≈ 2,6% dos 32 KiB reservados.

Resultados medidos

Teste oficial com 2.947 janelas de pessoas não vistas



81,13%

accuracy INT8

0,8070

macro F1

-0,136 pp

queda vs FP32

98,85%

mesma classe FP32 × INT8

Matriz de confusão INT8

linhas = classe real · colunas = classe prevista

	Caminhar	Subir	Descer	Sentado	Em pé	Deitado	Recall
Caminhar	423	52	21	.	.	.	85,3%
Subir	152	308	10	1	.	.	65,4%
Descer	82	39	299	.	.	.	71,2%
Sentado	.	2	.	342	146	1	69,7%
Em pé	1	4	.	45	482	.	90,6%
Deitado	.	.	.	.	.	537	100,0%

Leitura dos erros

- **Subir/descer → caminhar (152 e 82):** passadas parecidas; as 36 estatísticas não guardam ordem temporal nem frequência.
- **Sentado → em pé (146):** só muda a inclinação do tronco, sutil num sensor na cintura.
- **Deitado 100%:** a gravidade muda de eixo, e a média de total_acc capta isso.

Quantização: Keras FP32 = TFLite FP32 = 81,27% (conversão sem perda; Δprob. máx. $6,6 \times 10^{-7}$) → INT8 81,13%.

Métricas no interpretador TFLite desktop. No ESP32, a equivalência foi verificada pelo REPLAY (slide 8).

Deploy e demonstração no Wokwi

Do REPLAY reprodutível ao modo LIVE com sensor

REPLAY

Seis janelas reais do teste passam pela pipeline completa no ESP32-S3.
Resultado: 6/6 PASS.

LIVE

O MPU6050 simulado é lido por I2C. Em repouso, o sistema previu LAYING com alta confiança.

```
HAR: firmware ESP32-S3 / ANSI C
modo=REPLAY windows=6
replay WALKING_DOWNSTAIRS PASS
replay summary: passed=6 total=6 status=PASS
```

4.240 B

modelo

32 KiB

arena

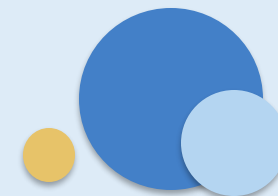
≈ 3,1 ms

Invoke no Wokwi

6/6 PASS = paridade numérica com o desktop nas janelas selecionadas. Não significa 100% de accuracy.

Conclusões e próximos passos

O que foi demonstrado e o que ainda precisa ser validado



O que o projeto comprova

- Pipeline completa de TinyML
- Modelo INT8 pequeno e funcional
- 81,13% de accuracy no UCI HAR
- 21 testes aprovados
- REPLAY 6/6 PASS no Wokwi
- Modo LIVE executando com MPU6050

Limitação principal

Existe diferença de domínio entre o smartphone do UCI HAR e o MPU6050. Por isso, a accuracy real do LIVE ainda não foi medida.

Próximo passo técnico

Coletar dados rotulados com o próprio MPU6050, recalibrar a normalização e reavaliar o modelo com pessoas independentes.

github.com/rodrigodondesenai/projeto-final-tinyml-har